

Tartarus Challenge

Vito Cafagno

April 30, 2026

1 INTRODUZIONE

La challenge è stata affrontata partendo dalla descrizione della composizione dei registri, tra loro contigui in memoria, della periferica da utilizzare. Inoltre, nello scenario veniva fornito il base address del primo di essi (FOO_PERIPH).

I registri (a 32 bit) della periferica sono tre, con le seguenti funzionalità:

1. **CTRL**: registro di "controllo" composto, in ordine, da 1 bit di **write**, uno di **read** e uno di **busy**, 8 bit per il **pin di trasmissione** e 8 per il **pin di ricezione**, 13 bit non usati (**RESERVED**).
2. **RXDATA**: il registro in cui la periferica scrive i dati per la MCU.
3. **TXDATA**: il registro da cui la periferica legge i dati scritti dalla MCU.

Il problema consisteva nel computare la media di 10 letture consecutive e inviare il risultato alla periferica.

2 SOLUZIONE

2.1 GESTIONE DELLA PERIFERICA

2.2 STRUTTURA DATI

Per modificare i registri in modo agevole è stato definito un nuovo tipo, una struttura dati composta da una struttura in cui venivano rappresentati i bit del registro di controllo della periferica, e due interi a 32 bit che rappresentavano i registri di RX e TX.

```
typedef struct{
    struct{
        uint32_t WRITE      :1;
        uint32_t READ       :1;
        uint32_t BUSY       :1;
        uint32_t TX         :8;
        uint32_t RX         :8;
        uint32_t RESERVED   :13;
    }__attribute__((packed)) CTRL;
    uint32_t RXDATA;
    uint32_t TXDATA;
}__attribute__((packed)) periphReg;
```

In entrambe le strutture si è preferito specificare l'attributo packed per il compilatore, per evitare errori di allineamento nella lettura/scrittura.

2.3 FUNZIONI PER L'USO

Sono state scritte tre funzioni per usare le funzionalità della periferica (inizializzare la periferica, effettuare la lettura dei dati scritti e inviare alla periferica la media). Rispettivamente:

```
uint8_t foo_periph_init(volatile periphReg* perifReg, uint8_t tx_pin, uint8_t rx_pin)
void foo_periph_tx_data(volatile periphReg* perifReg, uint32_t data)
uint32_t foo_periph_rx_data(volatile periphReg* perifReg)
```

Tra queste l'unica funzione un po' più complessa è quella di inizializzazione. Infatti, aspetta che il bit busy del registro di controllo sia 0, controlla che i pin siano nel range permesso per assegnarli ai bit del registro di controllo relativi alla TX e RX e, successivamente, imposta i control bit per lettura e scrittura a 0. Se tutto va a buon fine la funzione fa *return 0*; *return 1* se la periferica resta busy oltre il timeout; *return 2* se sono stati assegnati alla periferica dei pin fuori dal range (32-143).

Nelle altre due funzioni vengono, preventivamente, impostati a 0 entrambi i bit di read e write. Nel caso di una read della MCU, viene impostato il bit di read del registro di controllo a 1 e letto il registro RXDATA. Nel caso di una write della MCU, invece, viene scritto il registro TXDATA e poi il dato viene trasferito impostando il bit di write del registro di controllo a 1.

2.4 BUFFER

Per completezza è stata creata una libreria per usare un tipo buffer e salvare i dati in una struttura dati prima di calcolare la media. Il buffer è stato opportunamente semplificato per il problema. La sua interfaccia espone due funzioni:

```
uint8_t add(Buffer* buffer, int32_t value)
uint8_t reset(Buffer* buffer)
```

La funzione `add` serve ad allocare lo spazio contiguo in memoria, grande quanto la capacità del buffer, se nessun dato è presente; se il buffer è già inizializzato, scrive in coda i dati quando letti.

La funzione `reset`, invece, serve a cancellare tutti i dati presenti nel buffer, andando ad azzerare la dimensione (nelle prossime letture i dati saranno sovrascritti).

2.5 FIRMWARE

Nel firmware, come prima cosa, è eseguita l'inizializzazione della periferica. Se essa va a buon fine viene posta a *True* una flag che fa entrare il programma nel loop; altrimenti l'esecuzione si arresta e viene restituito l'errore che ha bloccato la periferica.

Una volta entrati nel loop, si controlla se la periferica è busy. Quando è libera si effettua la read e si mette il dato nel buffer. Se il buffer è pieno si calcola la somma dei dati salvati (il tipo della variabile che contiene la somma è `uint64_t` per evitare l'overflow), si divide per `capacity`, calcolando così *mea*, scritta nel registro TX dopo essersi assicurati che la periferica non sia busy. La variabile *mea*, essendo `uint32_t`, contiene solo i 32 bit bassi della variabile *sum*; questo, tuttavia, non è un problema perché sappiamo che la media di numeri a 32 bit è anch'essa un numero rappresentabile a 32 bit.

Una volta completata la scrittura, si svuota la coda.

L'unica attenzione da porre è che, nel caso peggiore, la media risulta troncata all'intero inferiore.